

Week-6 Assignment – Apache Spark

Name	Sanskruti Dnyaneshwar Shinde
College	Sanjivani College of Engineering, Kopargaon
CEI ID	CT_CSI_DE_1174

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Whenever we run a Spark application, there are basically three main players working behind the scenes. The **Driver** is like the brain of the whole application – it's the process where our `main()` function runs. It converts our code into a logical plan, then breaks that plan into stages and tasks, and it also keeps track of everything happening across the cluster. Then comes the **Cluster Manager** (it can be YARN, Mesos, Kubernetes, or Spark's own Standalone manager). Its job is simple – it takes care of resource allocation, meaning it decides which worker nodes will give how much CPU and memory to our application, and it launches the executors on our behalf. Lastly we have the **Executors**, which actually run on the worker nodes. These are the ones doing the real hard work – executing the tasks assigned by the driver, storing data in memory/disk for caching, and sending the results back to the driver. So basically, Driver plans everything, Cluster Manager arranges the resources, and Executors do the actual processing.

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

In Spark, when we write a chain of transformations like `filter`, `select`, `groupBy` etc., Spark doesn't execute them immediately. It just remembers what needs to be done by building a logical plan (like a to-do list), and only when we call an action such as `show()` or `collect()`, the actual execution starts. This is called **Lazy Evaluation**. The reason this improves performance a lot is because Spark gets a chance to look at the entire chain of operations together before running anything, and then it

optimizes the whole plan through its Catalyst optimizer – for example combining multiple filters into one pass, skipping columns we never use, or pushing filters closer to the data source. So instead of processing the full dataset again and again after every single line of code, Spark processes it only once in the most efficient way it can figure out. This saves a huge amount of time and memory, especially when we're dealing with large datasets.

Q3. Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read.csv("data/source.csv", header=True, inferSchema=True)
```

Q4. What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

CSV stores data **row by row**, so if we have 10 columns, every row's 10 values are stored one after another, and then the next row starts. Parquet, on the other hand, stores data **column by column** (columnar storage), meaning all values of a single column are stored together. This difference matters a lot for performance because in real projects we usually don't need all the columns for our query – we might only need 2 or 3 columns out of 20. With CSV, Spark still has to read the entire row (all columns) even if we only want one column, because there's no way to skip the rest. But with Parquet, since data is grouped by column, Spark can directly jump to just the columns we actually need and skip the rest entirely. On top of that, Parquet is compressed and stores the schema and metadata along with the file, so it takes less disk space and reads faster. That's basically why Parquet is preferred over CSV for big data workloads.

Q5. Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```
df.select("product_id", "price").filter(df.category == "Electronics").show()
```

Q6. Write the code to "revise" a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

```
from pyspark.sql.types import DoubleType

df = df.withColumnRenamed("old_name", "new_name") \
        .withColumn("price",
df["price"].cast(DoubleType()))
```

Q7. How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Spark keeps track of every transformation we apply on an RDD/DataFrame in the form of a **Lineage Graph**, also called the DAG (Directed Acyclic Graph). This graph basically records the whole history – which transformation was applied on which data, and in what order, starting right from the original source data. Now, if a worker node crashes or fails in the middle of a job and the data (partition) stored on that node is lost, Spark doesn't panic or ask us to restart the whole job. Instead, it just looks at the lineage graph, finds out exactly which transformations were needed to build that particular lost partition, and recomputes only that part on another available node. Since Spark isn't replicating the actual data everywhere (which would be expensive), and instead just remembers "how" to rebuild it, this makes the system fault-tolerant without wasting too much memory or storage.

Q8. Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

```
df_orders.filter((df_orders.status == "Completed") &
(df_orders.amount > 1000)).show()
```

Q9. Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Predicate Pushdown basically means pushing our filter condition down to the file-reading level itself, instead of first loading all the data into memory and then filtering it afterward. Since Parquet stores metadata like min/max values for each column chunk, when we write something like `df.filter(df.age > 30)`, Spark can actually check this metadata first and skip entire row groups or column chunks that clearly don't satisfy the condition, without even reading them from disk. So instead of loading the complete dataset into memory and then throwing away rows that don't match, Spark loads only the relevant data right from the start. This drastically reduces I/O and memory usage, especially for huge datasets, because we're not wasting resources bringing in rows that were going to get filtered out anyway.

Q10. Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).

```
df = df.withColumn("final_price", df["base_price"] * 1.18)
```

Q11. What is the difference between Transformations and Actions? Provide two examples of each.

Transformations are operations that create a new DataFrame/RDD from an existing one, but they are lazy – meaning they don't execute right away, Spark just adds them to the lineage/DAG and waits. **Actions**, on the other hand, are the ones

that actually trigger the execution of all the pending transformations and return a result back to the driver or write data to storage.

- **Transformations (examples):** filter(), select(), map(), withColumn(), groupBy()
- **Actions (examples):** show(), collect(), count(), write(), take()

So in short, transformations just build the plan, and actions are what actually make Spark do the work.

Q12. Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```
df = spark.read.parquet("path/to/input")
df_filtered = df.filter(df.user_id.isNotNull())
df_filtered.write.csv("path/to/output", header=True)
```

Q13. In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

In **Client Mode**, the Driver program runs on the machine from where we submitted the job (our local laptop or an edge/gateway node), and it stays outside the cluster, while only the executors run inside the cluster. This mode is generally used for interactive work, like when we're testing something or running a notebook, because we can see the output directly on our screen. But if our local machine disconnects or shuts down, the whole application dies since the driver was living there.

In **Cluster Mode**, the Driver itself runs inside the cluster on one of the worker nodes, managed by the cluster manager, and not on our local machine at all. This is what's mostly used in production, because once we submit the job, we can literally close our laptop and the job keeps running fine on the cluster since the driver isn't dependent on our local connection anymore.

Q14. Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
df.filter((df.region == "North") | (df.priority == "High")).show()
```

Q15. When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

When we call `.collect()`, Spark tries to pull **all** the data from every executor across the entire cluster and bring it back into the driver's memory as a single list. Now if the dataset is multiple terabytes in size, there is simply no way the driver (which usually has much less memory than the whole cluster combined) can hold all of that at once – it will most likely crash with an Out Of Memory error and bring the whole application down. On the other hand, `.show(5)` only asks Spark to compute and bring back just 5 rows for a quick preview, which is extremely lightweight and safe no matter how huge the dataset is. So for just exploring or peeking at the data, `.show(5)` is the smart and safe choice, and `.collect()` should only be used when we are absolutely sure the resulting data is small enough to fit in the driver's memory.